本数字教材特色

- 来源: https://class.imooc.com/lesson/5252#mid=104867
- 阶段: 阶段一: 【认知基石】理解大模型的本质与工作原理
- 周: 第1周 语言模型是什么
- 模块: 序与前言
- 课节: 图文 1-4 本数字教材特色

1-4 本数字教材特色

本数字教材特色

> [!TIP] 阅读指南 本节阐述本书的核心特色与学习理念，说明为何需要掌握大模型核心原理、程序员在AI时代的角色转变，以及本书的知识体系设计。 作为全书的开篇，帮助读者建立正确的学习预期与方法论。

2.1 为什么要掌握大模型的核心原理

工具仓库的地上散落着上百种工具：锤子、扳手、螺丝刀，也有电钻、切割机、焊接枪。每一个工具上都贴着陌生的标签：Prompt Engineering、Chain of Thought、RAG、Function Calling、Multi-Agent、LangChain……

工具都有用，但面对它们会生出疑问：这个工具是干什么的？什么时候该用它？它和其他工具有什么关系？应该先用哪个，后用哪个？

很多程序员在入门时，就是这样的感觉。概念太多，不清楚从哪里入手。

碎片化的信息容易引发焦虑：是否遗漏了重要内容？这些概念之间是什么关系？应该先学什么，后学什么？

把AI时代的所有核心概念重新深入梳理一遍，正是编写这本书的初衷。

2.2 AI时代程序员的转变

学习一门技术，过去的做法是从细节入手。学编程，要记住每个语法规则；学算法，要掌握每个数据结构；学框架，要熟悉每个API的用法。因为在那个时代，掌握细节就意味着掌握了能力。

AI时代改变了这个逻辑。

在编程细节层面，AI已经比人类做得更好。

这意味着程序员的工作重心正在转移。过去，价值体现在"如何实现"——你能多快写出一个排序算法，多熟练地掌握一个框架的API。现在，这些细节AI都能完成，价值转移到了以下两点：

- “实现什么”，这由你的认知和想象力决定
- “如何组合”，这由你的工程能力决定

“实现什么”：认知决定上限

AI能写出优雅的代码，但它不知道该写什么。它不知道用户的痛点在哪里，不知道业务的瓶颈是什么。

这些判断来自于对行业的理解、对用户的洞察、对技术的敏感度。一个见过多种架构模式的程序员，能看出当前系统的局限；一个了解多种AI能力的开发者，能想到创新的应用场景。

认知的高度，决定了你能看到多大的可能性。

“如何组合”：工程能力决定落地

知道要做什么还不够，还需要知道怎么做成。AI能完成单个任务，但把多个技术组合成一个完整的系统，需要工程能力：

- RAG提供知识检索，Function Calling提供外部调用，Agent提供自主决策——什么时候用哪个？
- LangChain适合快速原型，LangGraph适合复杂流程——如何选择合适的框架？
- 多个Agent如何协作？数据如何流转？错误如何处理？

这些不是API调用的问题，而是系统设计的问题。需要理解每个组件的能力边界，知道如何把它们拼装成一个可靠的整体。

过去，工程能力体现在代码的细节实现上。现在，体现在技术的组合架构上。

2.3 完整的知识体系

这本书的目的不只是教授每个技术的具体用法。它更想提供的，是一套完整的知识体系。

一套将AI领域70+核心概念、20+主流框架工具，按照能力递进的逻辑有序组织起来的体系。这些技术不是孤立的点，而是一条相互关联、层层递进的主线。

这套体系包含两个层面：

- 9个演进阶段：从最基础的大模型原理，到现在最流行的A2A多智能体协作，了解每个技术的原理、价值以及来龙去脉
- 3个方面：原理、工具、开发，从3个角度让理论知识扎实

有了这套体系，能理解技术之间的关联：

- Prompt Engineering为什么在RLHF之后？因为对齐技术让模型更听话，但Prompt能更精准地表达意图
- RAG为什么在CoT之后？因为模型会推理了，但缺乏事实，RAG提供知识支持
- LangChain和LangGraph有什么区别？它们不是对立的，而是互相辅助的

当未来出现新技术时，能够迅速判断：

- 它解决的是哪个阶段的问题？
- 它在整个演进链条中的位置在哪里？
- 它和现有技术是什么关系？

知识是成体系的。理解了技术之间的关系，才能真正掌握AI开发的全貌。

本书字数总计434K，分布如下：

2.4 兼具原理与实战的好书

不会浮于表面地介绍各种AI概念，然后让读者自己去摸索。在关键的技术上，会深入到具体的使用方案和编码细节，教如何应用各种框架和技术。

例如：

- 讲RAG时，不只讲概念，还会展示如何用LangChain或LlamaIndex搭建一个实际的知识库检索系统
- 讲Agent时，会带读者从零开始构建一个能自主完成任务的智能体
- 最后，还有一个大实战，教如何用CrewAI和知识图谱开发一个多智能体协同项目

这本书兼具概念讲解与实践应用两个层面：

- 概念层面：提供清晰的脉络和扎实的理论，理解技术的来龙去脉

- 实践层面：提供可执行的代码和方案，真正会用这些技术

目标不是让读者成为概念的收藏家，而是真正能用AI解决实际问题的开发者。

2.5 本书的脉络

本书共12章（另加第0章序言），构建了从「理解语言模型」到「构建多智能体系统」的完整学习路径，按能力递进分为六个阶段：

第一阶段：理解语言模型（第1-3章）

从最基础的问题出发：大模型到底是什么。

第1章从「文字接龙」的视角揭示了大模型的本质——它不是魔法，而是基于概率的文本生成工具。理解Token机制、参数演化和模型结构。

第2章深入训练过程：Embedding如何把文字变成数学向量，预训练如何从海量数据中学习规律，监督微调如何将通用模型适配到特定任务。

第二阶段：初入大模型实战（第4章）

第4章完成从理论到实践的跨越。学会调用API、管理多轮对话、控制流式输出、设计精密Prompt。从简单的Hello World到专业翻译器，第一次体验大模型开发的完整工作流。

这一章的核心理念是：AI不等于大模型。真正的开发是将大模型作为组件，融入工程化思考。

第三阶段：赋予AI知识与能力（第5-7章）

这是AI从「会说话」到「会做事」的关键转变。

第5章通过RAG技术解决了大模型的「知识过时」问题——不再依赖静态训练数据，而是从向量数据库中动态检索最新知识，同时通过文档分块、检索优化等技巧提升回答质量。

第6章通过Function Calling让LLM能够调用外部工具——查询数据库、调用API、操作文件系统。AI第一次具备了对现实世界的影响力。

第7章引入MCP协议，这是工具调用的「USB标准」。它将纷繁多样的工具接口统一为标准化协议，让开发者不再为每个平台和模型编写适配代码。

第四阶段：用框架提升效率（第8-9章）

第8章介绍了LangChain——它解决了重复造轮子的问题，通过Chain机制串联多个处理步骤，用Memory管理对话上下文，让开发者聚焦业务逻辑而非底层调用。

第9章的LangGraph则将AI应用从「单次调用」升级为「有状态的工作流」。StateGraph让流程可视化，条件分支实现动态决策，Human-in-the-Loop让人机协作更加灵活。

第五阶段：能力封装（第10章）

第10章引入了Skills——将专业知识打包成可复用的AI能力模块。学会如何通过YAML配置和触发机制，将零散的Prompt和工具调用封装为团队可共享的标准化技能。前端设计实战展示了Skills在真实场景中的价值。

第六阶段：智能体构建（第11-12章）

这是全书的技术顶峰。第11章从Agent的核心架构出发——感知、规划、执行、记忆——剖析了智能体的本质工作机制。理解ReAct推理模式如何将思考与行动循环结合，以及从Function Calling到Agent的自然演进。

第12章将视野从单个Agent扩展到多Agent协作。CrewAI框架的角色分工与任务协作模式，A2A协议实现跨平台Agent互操作。两个实战项目——题目生成器和考试系统——让你亲手构建从设计到实现的完整多Agent系统。

每一章都在为下一章铺路：Function Calling是Agent的工具能力基础，RAG是Agent的知识来源，LangGraph是Multi-Agent的编排框架。学完第12章，回头看第1章，会发现那个「基于概率的文字接龙工具」，已经演化成了一个能自主决策、多Agent协作的智能系统。

这本书不只是一本技术手册。它试图呈现一种理解方式——如何在快速变化的技术浪潮中，抓住那些不变的本质。

技术从来不会突变，从农业时代到信息时代，科技的进步总是有迹可循。理解了这个逻辑，即使以后再有新的技术出现，也可以轻松的入门。

2.6 不懂数学也能看懂大模型基础理论

这是一本为程序员写的AI书，不是数学教材。

很多AI书籍让人望而却步，翻开第一页就是密密麻麻的数学公式：矩阵运算、梯度下降、反向传播……仿佛不先读完《线性代数》《概率论》就没法理解AI。

本书不同。在理论深度与实用性之间找到了一个巧妙的平衡点：

用类比代替公式

需要解释复杂概念时，不会直接抛出数学公式，而是从生活中寻找恰当的比喻：

- Transformer的自注意力机制 → “像在会议中，你会自动重点关注某些发言人”

- 梯度下降 → “蒙着眼睛下山，每次用脚感受坡度，朝下坡方向走一小步”
- Embedding向量化 → “给每个词分配一个GPS坐标，相似的词距离更近”

这些类比不是为了回避原理，而是让读者先建立直觉理解，再逐步深入技术细节。

只保留最必要的基础数学

本书也会出现一些必要的数学表达，但遵循"够用就好"的原则，大致只需要初高中的数学水平即可理解：

不会出现的：

- 复杂的矩阵求导推导过程
- 冗长的优化算法证明
- 深奥的概率论公式链

会出现的：

- $y = f(x, \theta_1, \theta_2, ..., \theta_n)$ ← 帮你理解"函数+参数"的本质
- 余弦相似度计算 ← 理解向量检索的核心思想
- Loss函数的概念 ← 理解模型如何"知道自己错了"

每当出现数学符号时，都会：

- 先用自然语言解释它要表达什么
- 配合图示或代码示例
- 强调"理解原理比严格证明更重要"

> [!TIP] 数学里有一个惯例，在构建理论时，可以将很多"公理"作为无需证明的基石被直接使用。比如欧几里得几何学中的"过两点有且仅有一条直线"，我们直接使用它，而不去质疑为什么。同样的，在学习AI时，你完全可以接受"Self-Attention能捕捉长距离依赖关系"这个结论，然后直接用它解决问题，而不必纠结背后的矩阵运算细节。这个思想对于数学基础不好的同学来说非常重要。

用代码验证理论

对程序员来说，代码比公式更直观。所以本书的每个核心概念，都配有可运行的代码：

- 不仅告诉读者Embedding是什么，还给出完整代码调用API生成向量

- 不仅讲解RAG原理，还手把手带读者构建一个真实的问答系统
- 不仅介绍Agent理论，还用CrewAI实现一个Multi-Agent协作项目

当你能运行代码、看到结果时，理论就不再抽象了。

2.7 你需要什么基础?

最低要求：

- 会一门编程语言（Python最佳，其他语言也可以，只是本书是用Python做演示）
- 理解基本的编程概念（函数、对象、API调用）
- 愿意动手运行代码

不需要：

- 机器学习背景
- 深度学习理论
- 高等数学基础
- 论文阅读能力

教材的承诺：

- 用最朴素的语言讲清楚复杂的概念
- 用类比、图示、代码让抽象理论具象化
- 用完整项目展示技术的真实应用
- 用渐进式披露控制学习曲线

2.8 源码在哪里?

本教材的案例源码对于学习非常重要，它不是可有可无的。购买课程后，在课程资料区可下载资料。

建议第一时间下载并结合书本配套阅读、运行。

2.9 答疑

两个途径来解答读者的疑问。

大模型辅助答疑

这是最好的方式。大模型是一个极其强大的"老师"，任何问题都可以直接问它，回答得既完整又专业，还具有扩展性。比如DeepSeek、豆包、GLM都可以。

> [!TIP] 大模型的首次回答，未必符合要求，这个时候一定要追问。当然很多同学不会提问，没关系，看完本书后会学习到很多的提问技巧。

作者答疑

也可能有些非技术疑问、勘误等，大模型不能很好地回答。这个时候可以访问答疑社区。

2.10 下一节预告

本节从学习理念出发，阐述了本书的知识体系设计和编排逻辑。下一节是本书的前言，将从 Vibe Coding 的视角切入，展示 AI 如何改变编程方式——从写代码到表达意图。

图片

📖 每章字数明细

章节	字数	约合
第0章 序与前言	13,468	13千字
第1章 语言模型是什么	38,014	38千字
第2章 语言模型怎么训练	23,232	23千字
第3章 语言模型的进阶能力	36,964	37千字
第4章 理解大模型开发	52,606	**53千字**
第5章 RAG：知识接入技术	67,096	**67千字**
第6章 Function Calling	21,882	22千字
第7章 MCP：模型上下文协议	46,093	**46千字**
第8章 LangChain框架入门	16,446	16千字
第9章 LangGraph：工作流编排	18,602	19千字
第10章 Skill：可复用技能封装	21,874	22千字
第11章 Agent：智能体架构	41,024	**41千字**
第12章 Multi-Agent：多智能体协作	36,424	36千字

总量约 **434 千字**，其中内容最重的三章：第5章 RAG（67千）、第4章 大模型开发（53千）、第7章 MCP（46千）